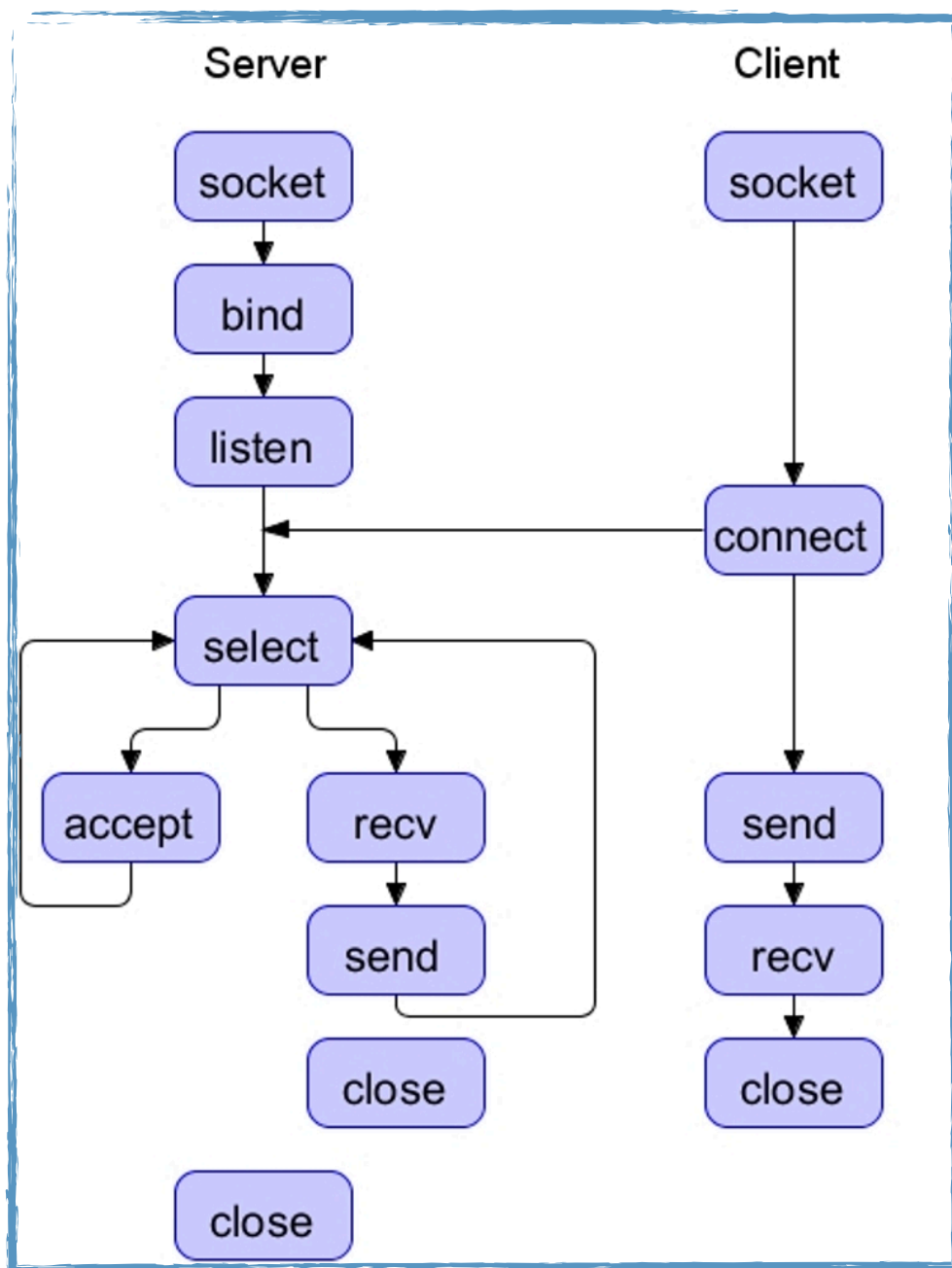


Laboratorul 9

Server TCP cu multiplexare I/O via primitiva select.
Exemplu de implementare.

1. Modelul Client/Server TCP



2. Utilizarea primitivei **select**

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
```

```
int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
struct timeval *timeout);
```

Parametri:

- **nfd** - descriptorul cu cea mai mare valoare din cele trei multimi de descriptori, la care se aduna 1;
- **readfds** - multimea de descriptori de citire care sunt monitorizati;
- **writefds** - multimea de descriptori de scriere care sunt monitorizati;
- **exceptfds** - multimea de descriptori de exceptie care sunt monitorizati;
- **timeout** - cat timp ar trebui sa blocheze primitiva select asteptand ca un descriptor sa devina "ready" pentru operatia I/O pe care este inregistrat;

Avem patru macrouri pentru a manipula multimile de descriptori:

- **FD_SET**(int fd, fd_set *set); // **Adauga descriptorul fd la multimea de descriptori set.**
- **FD_CLR**(int fd, fd_set *set); // **Remove fd from the set.**
- **FD_ISSET**(int fd, fd_set *set); // **Return true if fd is in the set.**
- **FD_ZERO**(fd_set *set); // **Clear all entries from the set.**

In caz de **succes**, select() returneaza numarul de descriptori continuti in cele trei multimi de descriptori (readfds, writefds si exceptfds), ceea ce ar putea fi 0 daca timeout-ul expira si nimic nu se intampla.

In caz de **eroare**, select() returneaza -1 si errno este setat astfel incat sa indice eroarea; multimile de descriptori nu sunt modificate, iar timeout devine undefined.

- **select()** - primitiva care permite unui program sa **monitorizeze** multipli descriptori, **asteptand pana cand unul sau mai multi descriptori devin "ready" pentru o anumita clasa de operatii I/O**. Un descriptor este considerat "ready" daca este posibil sa executam operatia I/O corespunzatoare.
- cu ajutorul primitivei select() urmarim trei clase de descriptori:
 - * descriptorii din multimea **readfds** vor fi urmariti astfel incat sa stim daca sunt disponibile caractere pentru citire;
 - * descriptorii din multimea **writfds** vor fi urmariti daca este disponibil spatiu pentru scriere;
 - * Descriptorii din multimea **exceptfds** vor fi urmariti pentru conditii exceptionale (spre ex. date OOB)
- fiecare din aceste multimi de descriptori poate fi specificata ca NULL daca nu vrem sa urmarim niciun descriptor pentru acele evenimente I/O.

3. Implementarea unui server TCP cu multiplexare I/O via primitiva select

Pentru a vizualiza un **exemplu de implementare a unui server TCP/IP cu multiplexare I/O via select** accesati urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/servTcpCSEL.c>

Pentru a descarca fisierul utilizati comanda **wget** intr-o sesiune activa de terminal, in urmatorul mod:

wget <https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/servTcpCSEL.c>

Pentru a vizualiza un **exemplu de implementare a unui client TCP/IP** accesati urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/cliTcp.c>

Pentru a descarca fisierul utilizati comanda *wget* intr-o sesiune activa de terminal, in urmatorul mod:

wget <https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/cliTcp.c>

Pentru a **compila** cele doua surse C, veti folosi un **Makefile** accesibil la urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/Makefile>

Pentru a descarca fisierul utilizati comanda *wget* intr-o sesiune activa de terminal, in urmatorul mod:

wget <https://profs.info.uaic.ro/~computernetworks/files/NetEx/S9/Makefile>